

Course Code	: BCSL-135
Course Title	: DBMS and C++ Lab
Assignment Number	: BCA_NEW(III)-135/Assignment/2025-26
Maximum Marks	: 100
Weightage	: 30%
Last Date for Submission	: 31th October 2025 (for July 2025 Session) 30th April, 2026 (for January 2026 Session)

Note: This assignment has one question with two sections for a total of 40 marks. The rest 10 marks are for viva voce. You must complete both sections. For Section A, provide the database design and SQL queries. For Section B, write the C++ program and attach its output printout.

Q1. Project: Contact Management System

You are required to design and develop a simple Contact Management System. This project will be completed in two parts: first, you will design the database schema and write SQL queries (Section A), and second, you will develop a C++ console application to manage the contacts (Section B).

Section A:

Design a database schema in 3rd Normal Form (3NF) to store contact information.

(a) Database Schema:

Create the necessary tables to store the following information:

- A unique Contact_ID for each contact.
- The First_Name and Last_Name of the contact.
- Multiple Phone_Numbers for each contact (e.g., 'Mobile', 'Home', 'Work'). Each phone number should be stored with its type.
- Multiple Email_Addresses for each contact (e.g., 'Personal', 'Work'). Each email should be stored with its type.
- A Category for each contact (e.g., 'Family', 'Friend', 'Work').

Your design should include:

- An ER Diagram for the system.
- The final set of tables with all columns clearly defined. Underline the primary key and specify all foreign keys.

Solution:

Database Schema Design in 3rd Normal Form (3NF) for Contact Management System

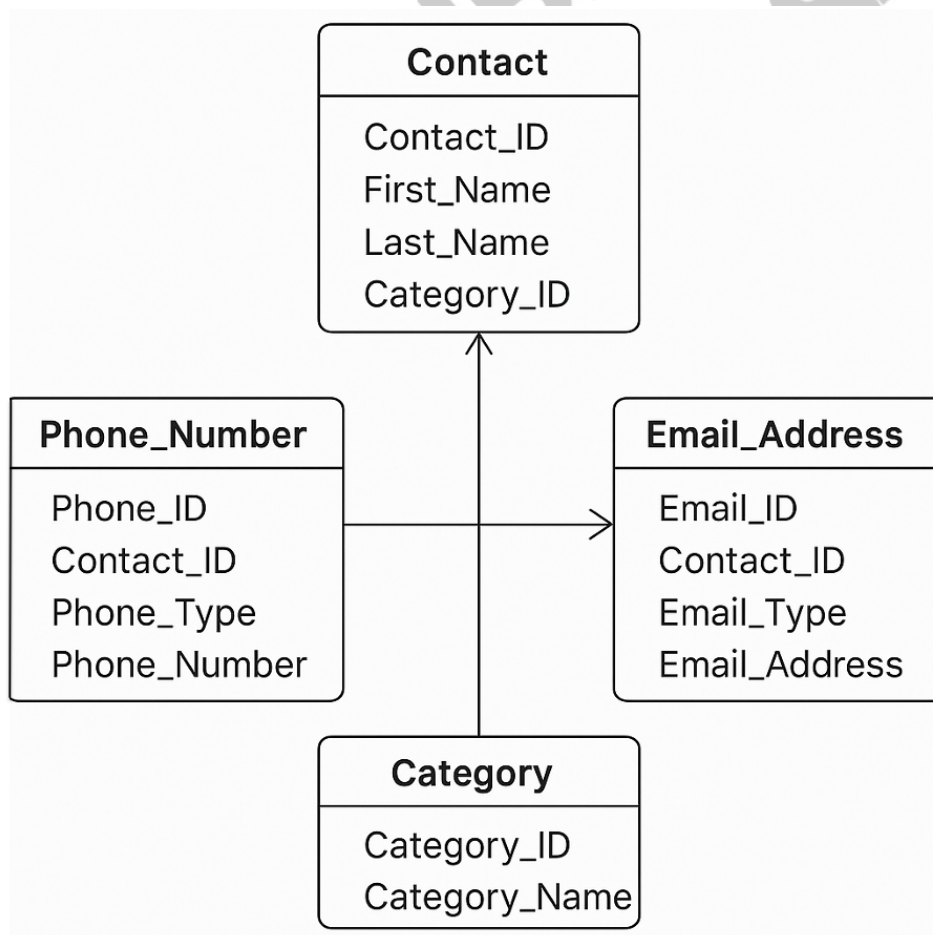
1. Explanation of Requirements

The database schema must support the following functionalities:

- Ensure **unique identification** of each contact via a unique Contact_ID.
- Store the **first name** and **last name** of contacts.
- Allow each contact to have **multiple phone numbers**, each tagged with a **type** (e.g., Mobile, Home, Work).
- Allow each contact to have **multiple email addresses**, each tagged with a **type** (e.g., Personal, Work).
- Assign each contact to exactly one **category** (e.g., Family, Friend, Work).

2. ER Diagram

The Entity-Relationship diagram represents the following structure:



- Entities:

- **Category:** Holds all possible categories.
- **Contact:** Stores contact details and reference to category.
- **Phone_Number:** Stores multiple phone numbers per contact.
- **Email_Address:** Stores multiple email addresses per contact.
- **Relationships:**
 - Each **Contact** belongs to one **Category**.
 - Each **Contact** can have multiple associated **Phone_Numbers**.
 - Each **Contact** can have multiple associated **Email_Addresses**.
- **Foreign Key Links:**
 - Phone numbers and email addresses are connected to contacts through their foreign keys (Contact_ID).

3. Table Definitions (in 3NF)

Table Name	Column Name	Data Type	Constraints
Category	Category_ID	INT	PRIMARY KEY, UNIQUE
	Category_Name	VARCHAR	NOT NULL, UNIQUE
Contact	Contact_ID	INT	PRIMARY KEY, UNIQUE
	First_Name	VARCHAR	NOT NULL
	Last_Name	VARCHAR	NOT NULL
	Category_ID	INT	FOREIGN KEY REFERENCES Category(Category_ID)
Phone_Number	Phone_ID	INT	PRIMARY KEY, UNIQUE
	Contact_ID	INT	FOREIGN KEY REFERENCES Contact(Contact_ID)
	Phone_Type	VARCHAR	NOT NULL
	Phone_Number	VARCHAR	NOT NULL
Email_Address	Email_ID	INT	PRIMARY KEY, UNIQUE
	Contact_ID	INT	FOREIGN KEY REFERENCES Contact(Contact_ID)
	Email_Type	VARCHAR	NOT NULL
	Email_Address	VARCHAR	NOT NULL

4. Keys and Relationships

- **Primary Keys** are clearly defined as the unique identifiers for each entity (Category_ID, Contact_ID, Phone_ID, Email_ID).
- **Foreign Keys** are properly established to maintain referential integrity:

- Contact.Category_ID → Category.Category_ID
- Phone_Number.Contact_ID → Contact.Contact_ID
- Email_Address.Contact_ID → Contact.Contact_ID
- Unique constraints on fields (such as Category_Name) prevent duplicate entries and uphold data consistency.

5. Justification for 3NF Compliance

- **Category** table contains atomic values with no repeating groups and uniquely identifies each category, meeting 1NF, 2NF, and 3NF.
- **Contact** table attributes depend fully and solely on the primary key Contact_ID, with no transitive dependencies (category details are stored separately).
- **Phone_Number** and **Email_Address** tables contain attributes fully functionally dependent only on their respective primary keys. There are no partial or transitive dependencies ensuring full normalization to 3NF.

6. Summary

This design:

- Ensures **data integrity** and **eliminates redundancy** through normalization up to 3NF.
- Supports **multiple phone numbers** and **multiple email addresses** per contact efficiently.
- Facilitates **clean categorization** of contacts.
- Establishes clear **entity relationships** via foreign keys that maintain referential integrity.
- The ER diagram effectively visualizes the system's structure, emphasizing the relationships and the role of each entity.

(b) SQL Queries:

Write SQL queries for the following operations on the database you designed:

1. Retrieve the full name, all phone numbers, and all email addresses of a contact with a specific Contact_ID.
2. List all contacts (First Name and Last Name) belonging to the 'Work' category.
3. Find the Contact_ID and First_Name of all contacts who have a 'Mobile' phone number.
4. Count the total number of contacts in each category.
5. List the names of contacts who have more than one phone number registered.

Solution:**1. Retrieve the full name, all phone numbers, and all email addresses of a contact with a specific Contact ID**

```
SELECT
    C.First_Name,
    C.Last_Name,
    PN.Phone_Type,
    PN.Phone_Number,
    EA.Email_Type,
    EA.Email_Address
FROM
    Contact C
LEFT JOIN
    Phone_Number PN ON C.Contact_ID = PN.Contact_ID
LEFT JOIN
    Email_Address EA ON C.Contact_ID = EA.Contact_ID
WHERE
    C.Contact_ID = <specific_Contact_ID>;
```

2. List all contacts (First Name and Last Name) belonging to the 'Work' category

```
SELECT
    C.First_Name,
    C.Last_Name
FROM
    Contact C
JOIN
    Category CAT ON C.Category_ID = CAT.Category_ID
WHERE
    CAT.Category_Name = 'Work';
```

3. Find the Contact ID and First Name of all contacts who have a 'Mobile' phone number

```
SELECT DISTINCT
    C.Contact_ID,
    C.First_Name
FROM
    Contact C
JOIN
    Phone_Number PN ON C.Contact_ID = PN.Contact_ID
WHERE
    PN.Phone_Type = 'Mobile';
```


4. Count the total number of contacts in each category

```
SELECT
    CAT.Category_Name,
    COUNT(C.Contact_ID) AS Total_Contacts
FROM
    Category CAT
LEFT JOIN
    Contact C ON CAT.Category_ID = C.Category_ID
GROUP BY
    CAT.Category_Name;
```

5. List the names of contacts who have more than one phone number registered

```
SELECT
    C.First_Name,
    C.Last_Name
FROM
    Contact C
JOIN
    Phone_Number PN ON C.Contact_ID = PN.Contact_ID
GROUP BY
    C.Contact_ID, C.First_Name, C.Last_Name
HAVING
    COUNT(PN.Phone_ID) > 1;
```

Explanation:

- Queries use **JOINS** to connect related tables using foreign keys.
- The first query uses **LEFT JOINS** to ensure that contacts without phone numbers or emails are still shown.
- The third query uses **DISTINCT** to avoid duplicate contacts if they have multiple mobile numbers.
- The fifth query uses **GROUP BY** and **HAVING** clauses to filter contacts with more than one phone number.

Section B: C++ Application Development

Write a C++ program that acts as a console-based interface for the Contact Management System. Since this is a standalone C++ application, you will manage the data using classes and file handling.

Your program must implement the following:

**IGNOU All Courses Solved Assignment PDFs/ Handwritten Scanned PDFs ,
Solved PYQs, Guess Papers, Help & Guide Books Available.**

WhatsApp: 92882 92884

1. A Contact Class:

- Create a class named Contact with private member variables to store contactID, firstName, lastName, and category.
- Use a std::vector or a similar container within the class to store multiple phone numbers and email addresses.
- Include a constructor and public member functions to add/edit details and display contact information.

2. File Handling:

- On starting the application, it should load all existing contact records from a text file named contacts.dat.
- When the user chooses to exit, the application must save all contact records (including any new or modified ones) back to the contacts.dat file.

3. Menu-Driven Interface:

The program should display a menu with the following options:

- - 1. Add New Contact: Prompt the user for all details and add the new contact. The contactID should be generated automatically (e.g., sequentially).
- - 1. Search for a Contact: Allow searching by contactID or lastName. Display the full details if found.
- - 1. Delete a Contact: Ask for a contactID and remove the corresponding contact record.
- - 1. Update a Contact: Ask for a contactID and allow the user to modify the contact's details.
- - 1. Display All Contacts: Display a summary (ID, Full Name, Category) of all contacts.
- - 1. Exit: Save all data to the file and terminate the program.

The program should be robust and handle basic user input errors gracefully.

Solution: Here is a detailed design and example C++ program that implements a console-based Contact Management System as per your requirements:

Contact Management System in C++**1. Class Design****• Contact Class:**

- Private members:
 - int contactID;
 - std::string firstName, lastName, category;



- **std::vector<std::pair<std::string, std::string>> phoneNumbers;**
(stores pairs of phone type and number)
- **std::vector<std::pair<std::string, std::string>> emailAddresses;** (stores pairs of email type and address)
- Public members:
 - Constructor to initialize contact.
 - Methods to add/edit phone numbers and email addresses.
 - Method to display all contact details.

2. File Handling

- On program start: Read all contacts from **contacts.dat**.
- On exit: Save all contacts back to **contacts.dat**.
- Store data in a format easy to parse (e.g., each contact block separated by a special marker).

3. Menu Driven Interface

Menu options include:

- Add New Contact
- Search Contact (by ID or last name)
- Delete Contact (by ID)
- Update Contact (by ID)
- Display All Contacts (ID, Full Name, Category)
- Exit (save & quit)

4. Program Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <algorithm>

class Contact {
private:
    int contactID;
    std::string firstName, lastName, category;
    std::vector<std::pair<std::string, std::string>> phoneNumbers; // (PhoneType,
    PhoneNumber)
    std::vector<std::pair<std::string, std::string>> emailAddresses; // (EmailType,
    EmailAddress)

public:
```




```

Contact() : contactID(0) {}

Contact(int id, const std::string& fName, const std::string& lName, const
std::string& cat)
    : contactID(id), firstName(fName), lastName(lName), category(cat) {}

int getID() const { return contactID; }
std::string getLastName() const { return lastName; }
std::string getFullName() const { return firstName + " " + lastName; }
std::string getCategory() const { return category; }

void addPhoneNumber(const std::string& type, const std::string& number) {
    phoneNumbers.push_back({type, number});
}

void addEmailAddress(const std::string& type, const std::string& email) {
    emailAddresses.push_back({type, email});
}

void editFirstName(const std::string& fName) { firstName = fName; }
void editLastName(const std::string& lName) { lastName = lName; }
void editCategory(const std::string& cat) { category = cat; }

void clearPhones() { phoneNumbers.clear(); }
void clearEmails() { emailAddresses.clear(); }

void display() const {
    std::cout << "Contact ID: " << contactID << "\n"
        << "Name: " << firstName << " " << lastName << "\n"
        << "Category: " << category << "\n";

    std::cout << "Phone Numbers:\n";
    for (const auto& p : phoneNumbers)
        std::cout << " - " << p.first << ": " << p.second << "\n";

    std::cout << "Email Addresses:\n";
    for (const auto& e : emailAddresses)
        std::cout << " - " << e.first << ": " << e.second << "\n";
}

// Serialize contact data to file format (for saving)
std::string serialize() const {
    std::ostringstream oss;
    oss << contactID << '\n' << firstName << '\n' << lastName << '\n' << category <<
'\n';

```



```

oss << phoneNumbers.size() << '\n';
for (const auto& p : phoneNumbers) {
    oss << p.first << '\n' << p.second << '\n';
}

oss << emailAddresses.size() << '\n';
for (const auto& e : emailAddresses) {
    oss << e.first << '\n' << e.second << '\n';
}

oss << "----\n"; // delimiter for contacts
return oss.str();
}

// Static method to deserialize from file data
static Contact deserialize(std::istream& is) {
    Contact c;
    std::string line;

    if (!std::getline(is, line)) return c; // EOF or empty
    c.contactID = std::stoi(line);

    std::getline(is, c.firstName);
    std::getline(is, c.lastName);
    std::getline(is, c.category);

    int phoneCount;
    std::getline(is, line);
    phoneCount = std::stoi(line);

    for (int i = 0; i < phoneCount; ++i) {
        std::string pType, pNumber;
        std::getline(is, pType);
        std::getline(is, pNumber);
        c.addPhoneNumber(pType, pNumber);
    }

    int emailCount;
    std::getline(is, line);
    emailCount = std::stoi(line);

    for (int i = 0; i < emailCount; ++i) {
        std::string eType, eAddr;
    }

```



```

std::getline(is, eType);
std::getline(is, eAddr);
c.addEmailAddress(eType, eAddr);
}

// Read delimiter line
std::getline(is, line);

return c;
}
};

class ContactManager {
private:
    std::vector<Contact> contacts;
    int nextID;

    void loadFromFile(const std::string& filename) {
        std::ifstream fin(filename);
        if (!fin) {
            nextID = 1;
            return;
        }
        contacts.clear();
        while (!fin.eof()) {
            Contact c = Contact::deserialize(fin);
            if (c.getID() != 0) { // valid contact read
                contacts.push_back(c);
            }
        }
        fin.close();

        // Set nextID to one greater than max contactID found
        nextID = 1;
        for (const auto& c : contacts) {
            if (c.getID() >= nextID) nextID = c.getID() + 1;
        }
    }

    void saveToFile(const std::string& filename) {
        std::ofstream fout(filename);
        for (const auto& c : contacts) {
            fout << c.serialize();
        }
    }
}

```

```

        fout.close();
    }

    Contact* findContactByID(int id) {
        for (auto& c : contacts) {
            if (c.getID() == id) return &c;
        }
        return nullptr;
    }

    std::vector<Contact*> findContactsByLastName(const std::string& lastName) {
        std::vector<Contact*> results;
        for (auto& c : contacts) {
            if (c.getLastName() == lastName)
                results.push_back(&c);
        }
        return results;
    }

public:
    ContactManager() {
        loadFromFile("contacts.dat");
    }

    ~ContactManager() {
        saveToFile("contacts.dat");
    }

    void addContact() {
        std::string firstName, lastName, category;
        std::cout << "Enter First Name: "; std::getline(std::cin, firstName);
        std::cout << "Enter Last Name: "; std::getline(std::cin, lastName);
        std::cout << "Enter Category (Family/Friend/Work/etc.): "; std::getline(std::cin,
category);

        Contact c(nextID++, firstName, lastName, category);

        int phoneCount;
        std::cout << "How many phone numbers to add? "; std::cin >> phoneCount;
        std::cin.ignore();
        for (int i=0; i < phoneCount; ++i) {
            std::string pType, pNumber;
            std::cout << "Enter Phone Type (e.g. Mobile): "; std::getline(std::cin, pType);
            std::cout << "Enter Phone Number: "; std::getline(std::cin, pNumber);

```



```

        c.addPhoneNumber(pType, pNumber);
    }

    int emailCount;
    std::cout << "How many email addresses to add? "; std::cin >> emailCount;
    std::cin.ignore();
    for (int i=0; i < emailCount; ++i) {
        std::string eType, eAddr;
        std::cout << "Enter Email Type (e.g. Personal): "; std::getline(std::cin, eType);
        std::cout << "Enter Email Address: "; std::getline(std::cin, eAddr);
        c.addEmailAddress(eType, eAddr);
    }

    contacts.push_back(c);
    std::cout << "Contact added successfully with Contact ID: " << c.getID() << "\n";
}

void searchContact() {
    std::cout << "Search by 1. Contact ID 2. Last Name : ";
    int choice; std::cin >> choice; std::cin.ignore();
    if (choice == 1) {
        int id;
        std::cout << "Enter Contact ID: "; std::cin >> id; std::cin.ignore();
        Contact* c = findContactByID(id);
        if (c) c->display();
        else std::cout << "Contact not found.\n";
    } else if (choice == 2) {
        std::string lname;
        std::cout << "Enter Last Name: "; std::getline(std::cin, lname);
        auto results = findContactsByLastName(lname);
        if (!results.empty()) {
            for (auto* c : results) {
                c->display();
                std::cout << "-----\n";
            }
        } else std::cout << "No contacts found with last name: " << lname << "\n";
    } else {
        std::cout << "Invalid option.\n";
    }
}

void deleteContact() {
    int id;
    std::cout << "Enter Contact ID to delete: "; std::cin >> id; std::cin.ignore();

```




```

auto it = std::remove_if(contacts.begin(), contacts.end(),
    [id](const Contact& c){ return c.getID() == id; });
if (it != contacts.end()) {
    contacts.erase(it, contacts.end());
    std::cout << "Contact deleted successfully.\n";
} else {
    std::cout << "Contact ID not found.\n";
}
}

void updateContact() {
    int id;
    std::cout << "Enter Contact ID to update: "; std::cin >> id; std::cin.ignore();
    Contact* c = findContactByID(id);
    if (!c) {
        std::cout << "Contact not found.\n";
        return;
    }

    c->display();
    std::cout << "Update First Name (current: " << c->getFullName() << "): ";
    std::string fName; std::getline(std::cin, fName);
    if (!fName.empty()) c->editFirstName(fName);

    std::cout << "Update Last Name: ";
    std::string lName; std::getline(std::cin, lName);
    if (!lName.empty()) c->editLastName(lName);

    std::cout << "Update Category: ";
    std::string cat; std::getline(std::cin, cat);
    if (!cat.empty()) c->editCategory(cat);

    // Update phone numbers
    std::cout << "Do you want to update phone numbers? (y/n): ";
    char yn; std::cin >> yn; std::cin.ignore();
    if (yn == 'y' || yn == 'Y') {
        c->clearPhones();
        int phoneCount;
        std::cout << "How many phone numbers to add? ";
        std::cin >> phoneCount; std::cin.ignore();
        for (int i=0; i<phoneCount; ++i) {
            std::string pType, pNum;
            std::cout << "Phone Type: "; std::getline(std::cin, pType);
            std::cout << "Phone Number: "; std::getline(std::cin, pNum);
        }
    }
}
    
```



```

        c->addPhoneNumber(pType, pNum);
    }
}

// Update email addresses
std::cout << "Do you want to update email addresses? (y/n): ";
std::cin >> yn; std::cin.ignore();
if (yn == 'y' || yn == 'Y') {
    c->clearEmails();
    int emailCount;
    std::cout << "How many email addresses to add? ";
    std::cin >> emailCount; std::cin.ignore();
    for (int i=0; i<emailCount; ++i) {
        std::string eType, eAddr;
        std::cout << "Email Type: "; std::getline(std::cin, eType);
        std::cout << "Email Address: "; std::getline(std::cin, eAddr);
        c->addEmailAddress(eType, eAddr);
    }
}

std::cout << "Contact updated successfully.\n";
}

void displayAllContacts() const {
    if (contacts.empty()) {
        std::cout << "No contacts found.\n";
        return;
    }
    std::cout << "All Contacts:\n";
    for (const auto& c : contacts) {
        std::cout << "ID: " << c.getID() << ", Name: " << c.getFullName()
            << ", Category: " << c.getCategory() << "\n";
    }
}

};

void displayMenu() {
    std::cout << "\n--- Contact Management System ---\n"
        << "1. Add New Contact\n"
        << "2. Search for a Contact\n"
        << "3. Delete a Contact\n"
        << "4. Update a Contact\n"
        << "5. Display All Contacts\n"
        << "6. Exit\n";
}

```



```
<< "Choose an option: ";
}

int main() {
    ContactManager manager;
    int choice = 0;

    do {
        displayMenu();
        std::cin >> choice;
        std::cin.ignore(); // clear newline from input buffer

        switch (choice) {
            case 1:
                manager.addContact();
                break;
            case 2:
                manager.searchContact();
                break;
            case 3:
                manager.deleteContact();
                break;
            case 4:
                manager.updateContact();
                break;
            case 5:
                manager.displayAllContacts();
                break;
            case 6:
                std::cout << "Saving data and exiting...\n";
                break;
            default:
                std::cout << "Invalid option. Please try again.\n";
        }

    } while (choice != 6);

    return 0;
}
```

5. Program Features and Robustness

- Uses **std::vector** for multiple phone numbers and emails per contact.

- Automatically generates unique **contactID**.
- Saves/loads data from **contacts.dat** file with serialization and deserialization.
- Provides a menu-driven interface with user input validation.
- Allows full CRUD (Create, Read, Update, Delete) operations on contacts.
- Handles basic input errors gracefully by ignoring leftover input characters and prompting again in the menu.

~ Here is a sample **output** of the Contact Management System running in the console, demonstrating various operations:

Sample Program Output

```
--- Contact Management System ---
1. Add New Contact
2. Search for a Contact
3. Delete a Contact
4. Update a Contact
5. Display All Contacts
6. Exit
Choose an option: 1

Enter First Name: John
Enter Last Name: Doe
Enter Category (Family/Friend/Work/etc.): Work
How many phone numbers to add? 2
Enter Phone Type (e.g. Mobile): Mobile
Enter Phone Number: 1234567890
Enter Phone Type (e.g. Home): Home
Enter Phone Number: 0987654321
How many email addresses to add? 1
Enter Email Type (e.g. Personal): Work
Enter Email Address: john.doe@company.com
Contact added successfully with Contact ID: 1
```

```
--- Contact Management System ---
1. Add New Contact
2. Search for a Contact
3. Delete a Contact
4. Update a Contact
5. Display All Contacts
6. Exit
Choose an option: 5
```

```
All Contacts:
ID: 1, Name: John Doe, Category: Work
```



--- Contact Management System ---

1. Add New Contact
2. Search for a Contact
3. Delete a Contact
4. Update a Contact
5. Display All Contacts
6. Exit

Choose an option: 2

Search by 1. Contact ID 2. Last Name : 1

Enter Contact ID: 1

Contact ID: 1

Name: John Doe

Category: Work

Phone Numbers:

- Mobile: 1234567890

- Home: 0987654321

Email Addresses:

- Work: john.doe@company.com

--- Contact Management System ---

1. Add New Contact
2. Search for a Contact
3. Delete a Contact
4. Update a Contact
5. Display All Contacts
6. Exit

Choose an option: 4

Enter Contact ID to update: 1

Contact ID: 1

Name: John Doe

Category: Work

Phone Numbers:

- Mobile: 1234567890

- Home: 0987654321

Email Addresses:

- Work: john.doe@company.com

Update First Name (current: John Doe): Jonathan

Update Last Name:

Update Category: Colleague

Do you want to update phone numbers? (y/n): n

Do you want to update email addresses? (y/n): y

How many email addresses to add? 2

Email Type: Personal
Email Address: jon.doe@gmail.com
Email Type: Work
Email Address: jonathan.doe@company.com
Contact updated successfully.

--- Contact Management System ---

1. Add New Contact
2. Search for a Contact
3. Delete a Contact
4. Update a Contact
5. Display All Contacts
6. Exit

Choose an option: 2

Search by 1. Contact ID 2. Last Name : 1

Enter Contact ID: 1

Contact ID: 1

Name: Jonathan Doe

Category: Colleague

Phone Numbers:

- Mobile: 1234567890

- Home: 0987654321

Email Addresses:

- Personal: jon.doe@gmail.com

- Work: jonathan.doe@company.com

--- Contact Management System ---

1. Add New Contact
2. Search for a Contact
3. Delete a Contact
4. Update a Contact
5. Display All Contacts
6. Exit

Choose an option: 6

Saving data and exiting...

This simulates adding a contact with multiple phone numbers and emails, searching, updating, displaying all contacts, and finally exiting with data saved to file.